



Introduzione a Spring

Python

Spring è un framework open source per lo sviluppo di applicazioni Java, progettato per semplificare la creazione di software strutturato, modulare e facilmente manutenibile.

Nasce con l'obiettivo di ridurre la complessità tipica delle applicazioni Java Enterprise, fornendo un insieme di componenti che permettono di gestire in modo centralizzato oggetti, configurazioni, accesso ai dati, sicurezza, comunicazione HTTP e integrazione con servizi esterni.

Alla base di Spring troviamo il concetto di Inversion of Control (IoC): invece di lasciare alle singole classi il compito di creare e gestire direttamente gli oggetti di cui hanno bisogno, è il framework a occuparsi del loro ciclo di vita e delle loro dipendenze.

Gli oggetti gestiti da Spring vengono chiamati Bean e sono contenuti all'interno dell'ApplicationContext, che rappresenta il principale container IoC dell'applicazione.



Spring non è un singolo componente, ma un ecosistema modulare composto da diversi progetti che possono essere utilizzati in base alle necessità dell'applicazione.

Il nucleo centrale è Spring Framework, al quale si affiancano tecnologie come Spring Boot per la configurazione automatica, Spring Data per l'accesso ai database, Spring Security per autenticazione e autorizzazione e Spring MVC per la realizzazione di applicazioni Web e API REST.

Nelle moderne applicazioni Java Spring viene spesso utilizzato seguendo un'architettura a livelli, separando responsabilità come Controller, Service e Repository, il Controller gestisce le richieste provenienti dal client, il Service contiene la logica applicativa e il Repository comunica con il livello di persistenza e questa separazione favorisce il loose coupling, migliora la testabilità del codice e permette di modificare singole parti dell'applicazione senza influenzare direttamente l'intero sistema.



Differenze tra Spring e Spring Boot

Spring è il framework principale che fornisce gli strumenti fondamentali per Dependency Injection, gestione dei Bean, sviluppo Web, sicurezza e accesso ai dati.

Spring Boot nasce invece come estensione dell'ecosistema Spring per semplificare configurazione e avvio dei progetti, introducendo auto-configurazione, dipendenze starter e server embedded come Tomcat.

In pratica, con Spring tradizionale molte configurazioni devono essere definite esplicitamente, mentre Spring Boot applica configurazioni predefinite e permette di creare rapidamente applicazioni eseguibili e API REST con meno codice infrastrutturale.



Spring Boot e approccio “Convention over Configuration”

Uno dei concetti più importanti introdotti da Spring Boot è il principio di Convention over Configuration, cioè l'utilizzo di convenzioni e configurazioni standard per ridurre il numero di impostazioni che lo sviluppatore deve definire manualmente.

Se Spring Boot rileva determinate librerie nel progetto, può configurare automaticamente componenti compatibili, come datasource, server Web o sistemi di persistenza.

Questo velocizza lo sviluppo soprattutto nelle fasi iniziali, mantenendo comunque la possibilità di modificare manualmente le configurazioni quando l'applicazione richiede comportamenti più specifici.



Principali casi d'uso aziendali

Spring è molto utilizzato in ambito enterprise per sviluppare applicazioni Web, API REST, backend per applicazioni mobile, gestionali, sistemi bancari, piattaforme e-commerce e microservizi.

La sua integrazione con database relazionali, sistemi di autenticazione, code di messaggi e servizi cloud lo rende adatto a progetti complessi e distribuiti.

In particolare, Spring Boot viene spesso scelto per creare servizi backend indipendenti e facilmente distribuibili, mentre Spring Security e Spring Data vengono utilizzati rispettivamente per la gestione degli accessi e della persistenza dei dati.



Limiti di Spring

Il principale limite di Spring è la complessità dell'ecosistema, soprattutto per chi si avvicina per la prima volta allo sviluppo enterprise in Java.

L'elevato numero di moduli, annotazioni e meccanismi automatici può rendere difficile comprendere cosa avviene internamente nell'applicazione.

Inoltre, applicazioni Spring molto grandi possono richiedere una gestione attenta delle configurazioni, delle dipendenze e delle prestazioni, e in alcuni casi il consumo di memoria e i tempi di avvio possono risultare superiori rispetto a framework più leggeri.



Le 4 principali caratteristiche di Spring

1. Dependency Injection

Spring gestisce automaticamente le dipendenze tra le classi, evitando di creare manualmente gli oggetti con `new`. Questo riduce l'accoppiamento tra i componenti e rende il codice più semplice da modificare e testare.

```
1. @Service  
2. public class UserService {  
3.   private final UserRepository repository;  
4.  
5.   public UserService(UserRepository repository) {  
6.     this.repository = repository;  
7.   }  
8. }
```



In questo caso Spring fornisce automaticamente un oggetto `UserRepository` al costruttore di `UserService`.

Le 4 principali caratteristiche di Spring

2. Gestione tramite Bean

Gli oggetti principali dell'applicazione possono essere gestiti direttamente dal container Spring. Annotazioni come `@Component`, `@Service`, `@Repository` e `@Controller` permettono al framework di individuare e creare automaticamente questi oggetti.

```
1. @Component  
2. public class EmailService {  
3.  
4.   public void sendEmail() {  
5.     System.out.println("Email inviata");  
6.   }  
7. }
```



EmailService diventa un Bean Spring e può essere utilizzato come dipendenza in altre classi.

Le 4 principali caratteristiche di Spring

3. Sviluppo Web e API REST

Spring, attraverso Spring MVC e Spring Boot, permette di creare rapidamente applicazioni Web e servizi REST. Le richieste HTTP vengono associate direttamente a metodi Java tramite specifiche annotazioni.

```
1. @RestController  
2. public class HelloController {  
3.  
4.     @GetMapping("/hello")  
5.     public String hello() {  
6.         return "Hello Spring!";  
7.     }  
8. }
```



Una richiesta GET /hello esegue il metodo hello() e restituisce direttamente la risposta al client.

Le 4 principali caratteristiche di Spring

4. Ecosistema modulare

Spring permette di integrare diversi moduli senza dover costruire manualmente tutta l'infrastruttura dell'applicazione. È possibile aggiungere funzionalità per database, sicurezza, Web, test e microservizi utilizzando componenti specifici come Spring Data, Spring Security e Spring Web.

```
1. public interface UserRepository  
2.     extends JpaRepository<User, Long> {  
3. }
```

Con Spring Data JPA è sufficiente estendere `JpaRepository` per ottenere automaticamente operazioni come `save()`, `findById()`, `findAll()` e `deleteById()`.



Problematiche comuni

Tra le problematiche più comuni nello sviluppo con Spring troviamo una configurazione errata dei Bean, dipendenze non risolte correttamente, conflitti tra versioni delle librerie e problemi nella gestione del ciclo di vita dei componenti.

Sono frequenti anche errori legati a Spring Security, configurazioni database non corrette, eccezioni nelle API REST e difficoltà nel debugging dovute all'elevato livello di automazione introdotto dal framework.

Nei progetti più grandi diventano inoltre importanti la gestione delle transazioni, delle performance e della corretta separazione tra Controller, Service e Repository.



Posizioni lavorative associate

Le competenze Spring sono richieste soprattutto per ruoli come Java Developer, Backend Developer, Software Engineer e Full Stack Developer, dove il framework viene utilizzato per sviluppare servizi, API e logica applicativa.

In contesti più strutturati è comune anche nelle figure di Microservices Developer, Cloud Developer e Software Architect, che lavorano su sistemi distribuiti, integrazioni tra servizi e architetture enterprise.

Una buona conoscenza di Spring viene spesso affiancata a Java, SQL, REST API, Git, Docker, strumenti cloud e sistemi di CI/CD.



Consigli per iniziare

Per iniziare con Spring è consigliabile avere prima una buona conoscenza di Java OOP, interfacce, eccezioni, collezioni e gestione delle dipendenze con Maven o Gradle, secondo me, il percorso più semplice è partire direttamente da Spring Boot, creando piccoli progetti con Spring Initializr e concentrandosi inizialmente su @RestController, @Service, @Repository e Dependency Injection.

Successivamente è utile introdurre Spring Data JPA per il collegamento a un database e Spring Security per autenticazione e autorizzazione.

L'obiettivo iniziale non dovrebbe essere imparare tutte le annotazioni disponibili, ma comprendere bene come Spring crea gli oggetti, come li collega tra loro e come una richiesta attraversa Controller, Service e Repository.



Buon davante a tutti

